

EC-333 Microprocessor and Interfacing Techniques

Experiment # 1

Introduction to Assembly Language.

1. Objective/s:

- Introduction to Assembly Language
- Introduction to emu8086 environment
- Understanding of General Purpose Registers, Flags, Memory
- Microsoft Word
- Lab Task

2. Assembly Language

To understand Assembly Language we will first try to understand Machine Code and then see Assembly Language in contrast with Machine Code

Machine code is binary (1's and 0's) code that can be executed directly by the CPU. If you were to open a "machine code" file in a text editor you would see garbage, including unprintable characters.

Assembly code is plain-text and (somewhat) human read-able source code that has a mostly direct 1:1 analog with machine instructions. This is accomplished using mnemonics for the actual instructions/registers/other resources. Examples include things like JMP or MULT for the jump and multiplication instructions. Unlike Machine Code, the CPU does not understand Assembly Code.

3. 8086 Emulator

Emulator 8086 provides a virtual machine to emulate 8086-microprocessor based system and comes with an integrated 8086 assemblers. The emulator can run programs on a Virtual Machine, and emulate real hardware including screen and memory. It helps you program in assembly language. The source code is compiled by assembler and then executed on Emulator step-by-step, allowing you to watch registers, flags and memory while your program runs. Emulator runs programs on a Virtual PC, this completely blocks your program from accessing real hardware, such as hard drives and memory.

3.1. Registers and Memory

The first CPU in the Intel family is the 8086. The registers inside the 8086 are all 16 bits. They are split up into four categories: General Purpose, Index, Status & Control, and Segment.

3.1.1. General Purpose Registers

The four general purpose registers are the AX, BX, CX, and DX registers.

AX - accumulator, and preferred for most operations.

BX - base register, typically used to hold the address of a procedure or variable.

CX - count register, typically used for looping.

DX - data register, typically used for multiplication and division.

All of the general purpose registers can be treated as a 16 bit quantity or as two 8 bit quantities. The high byte is referenced by replacing the X with H. The low byte is referenced by replacing the X with L: For example, putting 0xF100 (the leading 0x indicates what follows is in hex) into AX is the same as putting F1 into AH, and then 00 into AL. We can reference DX in terms of DH, DL, CX in terms of CH, CL, and BX in terms of BH, BL as well.

3.1.2. Segment Registers

The CPU contains four segment registers, used as base locations for program instructions, data, or the stack. In fact, all references to memory on the IBM PC involve a segment register as a base location.

The registers are:

CS – Code Segment, base location of program code

DS – Data Segment, base location for variables

SS – Stack Segment. Base location of the stack

ES – Extra Segment. Additional base location for variables in memory.

3.1.3. Index Registers

The index registers contain offsets from a segment register for information we are interested about. There are four index registers:

BP – Base Pointer, offset from SS register to locate variables on the stack

SP – Stack Pointer, offset from SS register as to the location of the stack's top

SI – Source Index, used for copying strings, segment register varies

DI – Destination Index, used for destination for copying strings

3.2. Calculating Addresses

How are these registers used to calculate addresses? On the Intel 8086, there is a 20 bit address bus giving us the capability to address up to 1MB of memory. However, registers are only 16 bits long. This means we can only address 64K of memory using a 16 bit register, leaving the entire 1Mb of memory seemingly un-addressable.

To solve this problem, the Intel engineers designed the CPU so that it computes 20 bit *absolute* addresses by combining the segment with the offset (i.e. index). Segmented addresses are represented by:

BaseSegment:Offset

e.g.: 08F0:0010

Absolute addresses are represented directly by the hex representing the 20 bits:

e.g.: 0AB41

How do we compute an absolute address given a segmented address? Note that in the segmented address we have a total of 32 bits. This is an additional 12 bits than we really need! The scheme to compute the absolute address is to multiply the segment by 16 (i.e. slide it over four bits to the left) and then add the result to the offset.

E.g. given 08F1:0010, here the segment is 08F1, multiply by 16 or shift four bits / one hex digit to the left, inserting a zero on the right giving 08F10.

Now add to the offset:

08F10

+ 0010

08F20 Absolute address

By sliding the segment over by four bits, we get a total addressable space of 20 bits.

Note that there is **overlap** among segments:

08F10 08F00

+ 0000 + 0010

08F10 08F10

That is, 08F1:0000 and 08F0:0010 both reference the same absolute address.

Using this approach allows the CPU to access the 20 bit address bus using the 16 bit registers. A benefit of the scheme is that it becomes compact and easy to reference data within a single segment. Another benefit is that it allows programs to be loaded into any segment address without having to recalculate the individual addresses of variables – they are merely references as offsets from the segment. Finally, programs can access large data structures by gradually modifying the segment portion of the address to access large blocks of memory. In this case, the overlap feature of the segment becomes useful. A downside of the scheme, in addition to the complexity, is the confusion that can result if a program needs to access data beyond a 64K block of memory.

When we visit the 32-bit Intel processor architecture, this problem goes away and we have a much cleaner addressing system.

3.2.1. Status and Control Registers

The last set of registers is the Instruction Pointer (i.e. Program Counter) and the Flags:

IP – Instruction pointer, offset from the CS for the next instruction to execute

Flags – contains status flags

These flags are:

Direction: Used for block transfer of data, may be 0=up or 1=down

Interrupt: 1 = enabled, 0=disabled

Trap: Determines if the CPU should halt after each instruction. 1=on, 0=off.

Carry: Set if the last unsigned arithmetic operation had a carry

Overflow: Set if the last signed arithmetic operation overflowed

Sign: Set if the last arithmetic operation was negative. 1=negative, 0=positive

Zero: Set if the last arithmetic operation generated a result of zero. 1=zero

Aux Carry: Set when a carry from bit 3 to bit 4

Parity: Used for a parity check, number of 1's

3.2.2. Operating System and Memory

The 1Mb of accessible memory in the 8086 ranges from 00000 to FFFFF.

RAM occupies 0000 – BFFFF.

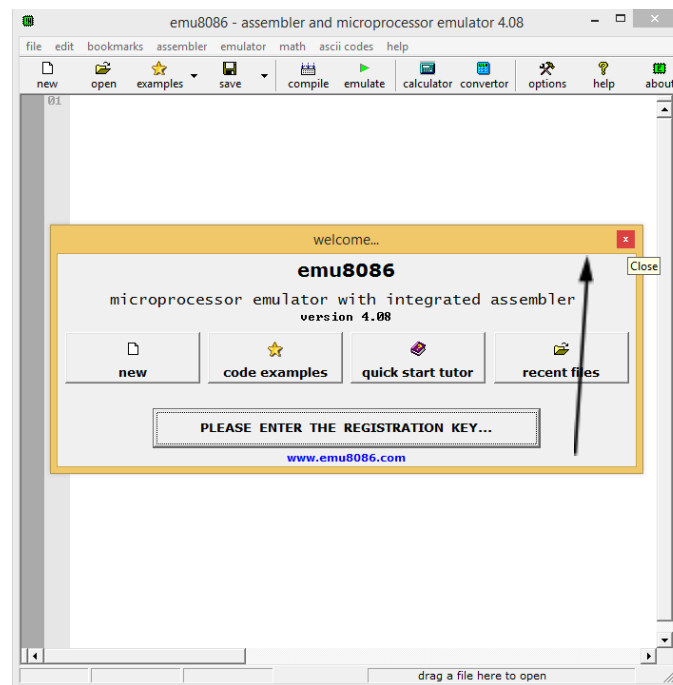
ROM occupies C0000 to FFFFF.

DOS uses the first 640K of memory, 0000 to 9FFFF.

3.3. Working with Emulator:

Step – 1:

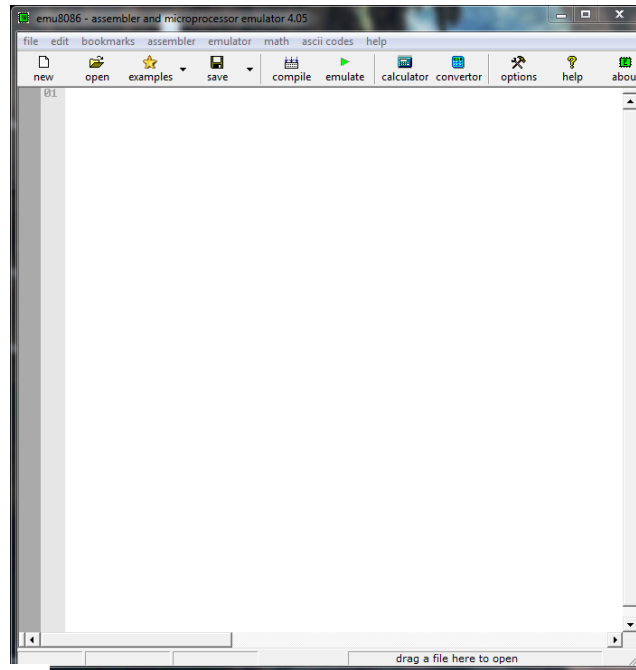
Once you open the emu8086 emulator you should see the following screen.



Step – 2:

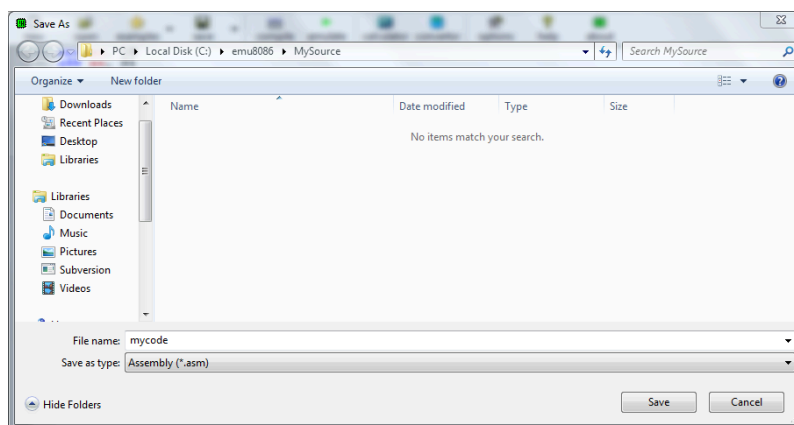
Click on the cross button a blank new page is opened as in the below figure.

From here on you can write your own code.



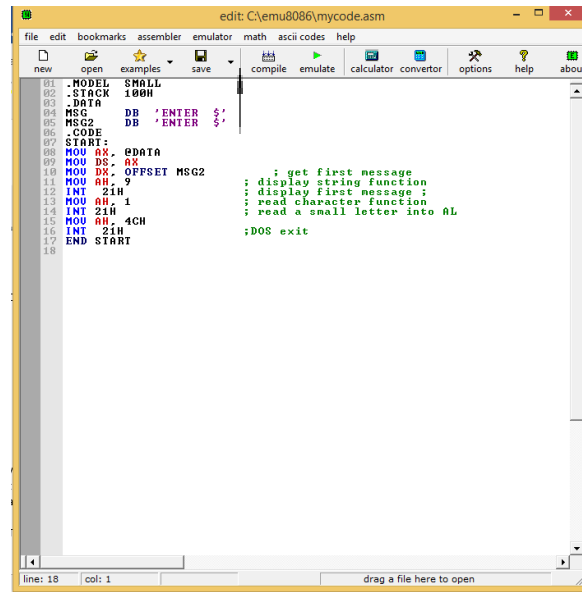
Step – 3:

Once you have typed in your program you should save the program by going to File>Save or File>save as or just by clicking the save button on the toolbar. Emu8086 will save it the newly created file with .asm extension in its default location (C:\emu8086\MySource) but you can browse to a folder where you want to save the file.



Step – 4:

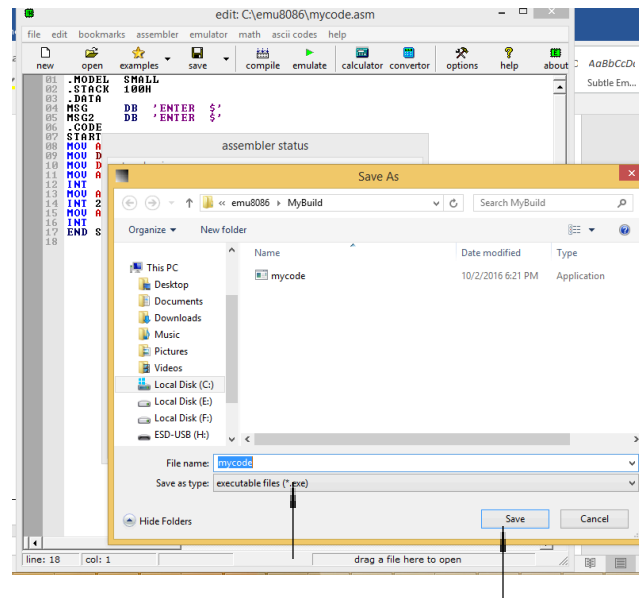
Once you have saved the file, you can continue building the executable file and running it. Click on compile as below:



```
edit: C:\emu8086\mycode.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04 MSG DB 'ENTER $'
05 MSG2 DB 'ENTER $'
06 .CODE
07 START:
08 MOV AX, @DATA
09 MOV DS, AX
10 MOV DX, OFFSET MSG2
11 INT 21H ; get first message
12 MOV AH, 9 ; display string function
13 INT 21H ; display first message
14 MOV AH, 1 ; read character function
15 INT 21H ; read a small letter into AL
16 MOV AH, 4CH ;DOS exit
17 INT 21H
18 END START
line: 18 col: 1 drag a file here to open
```

Step – 5:

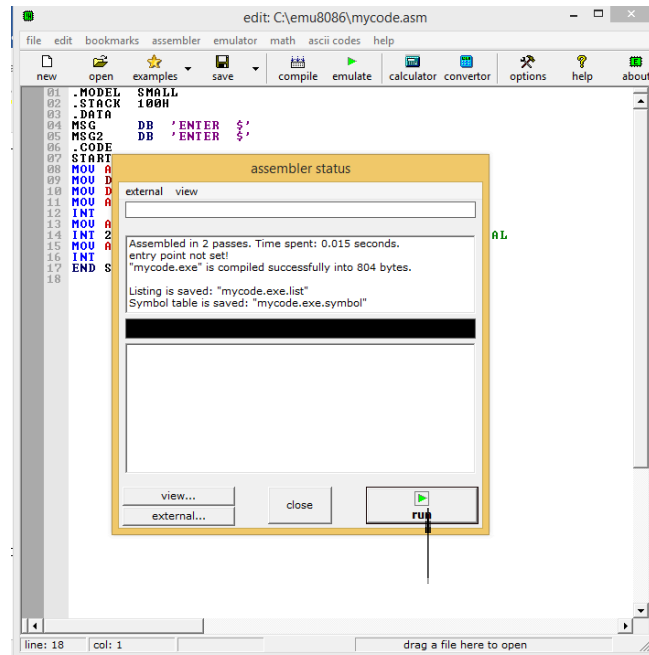
The following will appear:



Step – 6:

Click on save make sure in the Save as type box it is .exe.

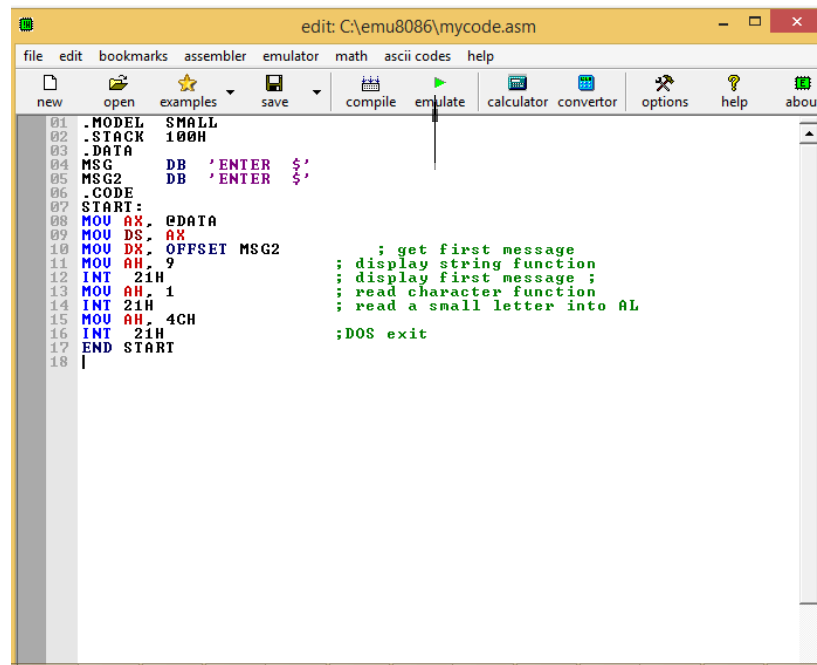
Next click on run as below:



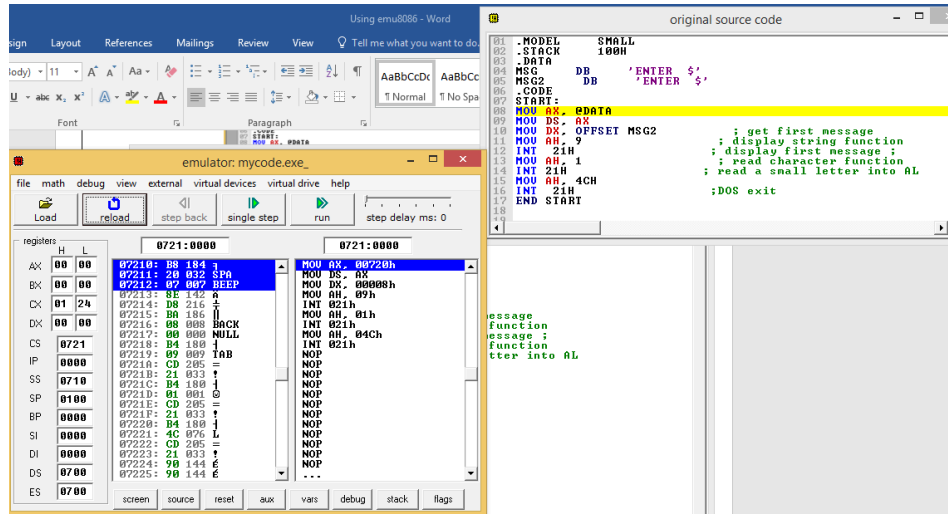
Step – 7:

The program will run.

Next you want to debug it. You can run it step by step. Click on the emulate button on the toolbar to see the behavior of the program as below:



The following window will appear:



4. Tasks

1. Write down the following code in emulator 8086:

```
01 org 100h
02 mov ax,100
```

Run the code step by step and observe the value changing. Answer the following questions:

- What is the value stored in register AX ?
- What are the value stored in AL and AH ?
- The values stored in AX is NOT hundred, Why?

2. Type the following code in your emulator 8086 environments:

```
01 .model small
02 .data ; Define variables in this portion
03 a db 15
04 b db 11
05 sum1 db 0
06
07 .code ; Write the normal code here
08
09 ; For now just add these two lines
10 ; The reason will be explained later
11 mov ax,@data
12 mov ds,ax
13
14
15 ; Now summing up two values
16 mov ah,a
17 mov al,b
18 add al,ah ; now al holds the sum
19
20 mov sum1,al
```

Run the code step by step and observe the value(s) changing. Answer the following questions:

- Write down the changes being occurred in each register with each step execution and tell why the values changes (for each register).
 - Locate the values stored in memory of the processor and observe changes before and after execution of instruction.
 - Show the values stored (in form of variables)
3. Write a Program that takes 10 numbers one by one from memory and adds them in a variable. In the end store the output in a new location in memory.
- Task 3 Reports is Evaluated in Report